
STRIVE: Auditing Execution Enabled Reasoning Agents Beyond Accuracy

Kshitij Dahiya¹
kshitij.03014802724@cse.mait.ac.in

Raaghav Kapoor¹
03314803125.raaghav@it.mait.ac.in

Saloni¹
13414803125.saloni@it.mait.ac.in

Vidit Kaushik¹
vidit.03914812723@cst.mait.ac.in

¹Maharaja Agrasen Institute of Technology

Abstract

Tool-using language agents are commonly ranked by final-answer accuracy, although deployment cost and failure often arise inside the interaction trace: an answer may be numerically correct yet unsupported by executed evidence, reflecting parametric recall or an unverified guess rather than evidence-based problem solving; on longer or computation-dependent tasks, this can produce brittle, silent failures. Fluent steps may move away from a solution, and most tokens may be spent before or after decisive evidence appears. Existing holistic model evaluations and interactive agent benchmarks expose important capabilities, but do not jointly operationalize correctness, executable provenance, step quality, token utility, reasoning-aware redundancy, reliability, and latency for sandboxed reasoning agents. We introduce STRIVE, an audit-first evaluation framework that separates these quantities instead of hiding them in one score. Correctness is resolved by deterministic equivalence with a selective judge fallback; grounding is a deterministic relation between the declared final answer and successful sandbox output; verified success is their conjunction. A hybrid process metric combines two process reward models (PRMs), execution signals, repetition checks, and a selectively invoked critic. Token utility is anchored to the first traceable solution point, and redundancy penalizes only repetition that neither advances nor verifies the solution. We evaluate six agents on a frozen set of 200 MATH-500 and 100 text-only OlympiadBench problems, yielding 1,800 attempts. The study reveals the correctness-grounding gap and ranking changes that accuracy alone conceals: MiniMax-M3 has the highest operational correctness (0.760) but only 0.390 verified success, whereas GPT-5 Nano reaches 0.610 verified success; GLM-5.2 is less accurate than MiniMax-M3 but requires the fewest tokens per verified solve (3,323). We release an auditable artifact containing trajectories, code, observations, evidence pointers, evaluator decisions, and analysis tables. The results support multidimensional reporting and Pareto analysis, not a universal scalar leaderboard.

1 Introduction

Language models increasingly act through interpreters, search systems, databases, and software environments. Interleaving reasoning with actions can improve problem solving Yao et al. [2023], Gao et al. [2023], Schick et al. [2023], yet it also changes what an evaluation must observe. A final answer does not reveal whether an agent used a tool correctly, whether the answer is traceable to the returned observation, when the solution became available, or how much computation was wasted along the way. For a sandboxed agent, these are measurable properties of an execution trace, not stylistic preferences.

This gap remains despite strong progress in evaluation. HELM argues for multi-metric evaluation of foundation models Liang et al. [2023]; AgentBench and MINT evaluate multi-turn tool interaction Liu et al. [2024], Wang et al. [2024b]; AgentBoard adds progress-oriented analysis Ma et al. [2024]; and environment-grounded benchmarks such as GAIA and SWE-bench test realistic agency Mialon et al. [2024], Jimenez et al. [2024]. These works are complementary, but none supplies a single provenance-aware protocol that jointly answers five operational questions for a sandboxed reasoning trajectory: (1) Was the declared answer correct? (2) Did executed evidence actually support that answer? (3) Did intermediate steps advance the solution? (4) Where were tokens useful or wasted? (5) Was repetition harmful or a legitimate verification? Treating these questions as one multiplicative “efficiency” score makes failures difficult to diagnose and model rankings sensitive to arbitrary weights.

STRIVE addresses this by making the execution trace the unit of audit. It records every model request, visible action, code block, sandbox observation, error, final answer, token component, and timing component. It then computes independent dimensions and reports their joint trade-offs. The framework is designed around four principles: deterministic checks before learned judgment; explicit missingness when a trace is unusable; operational denominators that retain failures; and trace-level evidence pointers that permit later re-evaluation.

Our contributions are:

- a correctness–grounding pipeline that distinguishes symbolic correctness, selective adjudication, provenance, and verified success;
- a reproducible process metric combining two PRMs with deterministic execution/error signals and a selective critic, plus an explicit uncertainty and ablation report;
- solution-point token accounting and reasoning-aware redundancy that do not punish useful verification;
- an artifact-backed evaluation of six agents on 300 problems (1,800 attempts), with coverage, confidence intervals, paired tests, budget curves, threshold sensitivity, and Pareto frontiers.

The empirical study is mathematical, but the evidence contract is domain-independent: replace symbolic equivalence and Python output with a domain verifier and typed evidence relation.

2 Related Work

2.1 Reasoning With External Computation

Chain-of-thought prompting elicits intermediate natural-language reasoning Wei et al. [2022], and self-consistency aggregates multiple paths Wang et al. [2023]. Program-aided methods instead delegate execution to an interpreter: PAL generates programs as intermediate reasoning Gao et al. [2023], Program of Thoughts separates computation from language reasoning Chen et al. [2023], and ReAct interleaves reasoning and environment actions Yao et al. [2023]. Toolformer studies learned API invocation Schick et al. [2023]. These methods motivate execution-aware evaluation, but their task scores generally do not establish that a reported final answer is the value supported by the executed observation. Moreover, a plausible verbal trace need not faithfully expose the cause of a model decision Turpin et al. [2023]; STRIVE therefore grounds claims in externally observed actions rather than treating prose as proof.

2.2 Holistic and Agent Evaluation

HELM formalizes broad, transparent, multi-metric model evaluation across scenarios Liang et al. [2023]. AgentBench evaluates agents in eight interactive environments Liu et al. [2024]; MINT studies tools and language feedback in multi-turn interactions Wang et al. [2024b]; and AgentBoard reports fine-grained progress in addition to success Ma et al. [2024]. GAIA targets questions requiring combinations of reasoning, multimodality, browsing, and tool use Mialon et al. [2024], while SWE-bench uses repository tests to evaluate software agents Jimenez et al. [2024]. These benchmarks establish the importance of interactive and multi-faceted evaluation. STRIVE differs in level of analysis: it is a metric layer over a fully recorded execution protocol, separating operational

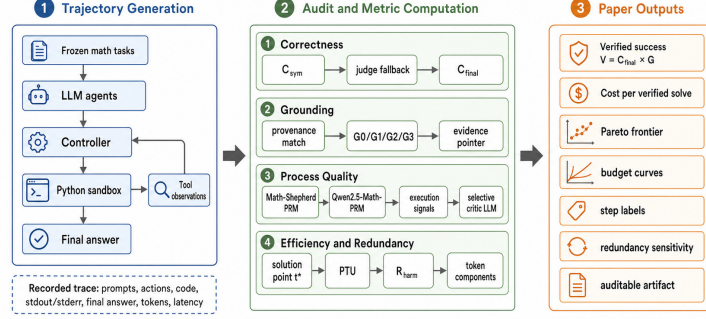


Figure 1: STRIVE evaluation pipeline from sandboxed trajectory generation to audit metrics and paper outputs.

success, provenance, process quality, token timing, and repetition for each trajectory. It can therefore complement, rather than replace, task suites.

2.3 Process Supervision and Automated Judges

Process supervision assigns feedback to intermediate steps rather than only to outcomes. PRM800K provides human step labels Lightman et al. [2023]; Math-Shepherd constructs process supervision automatically Wang et al. [2024a]; and Qwen2.5-Math develops mathematical reward models as part of a self-improvement pipeline Yang et al. [2024]. ProcessBench shows that locating the earliest erroneous step remains difficult and that PRMs may generalize poorly beyond their training distribution Zheng et al. [2025]. Consequently, STRIVE does not equate one PRM score with reasoning quality: it uses two PRMs, measures their agreement, incorporates deterministic failures and tool gains, and calls a critic only near a decision boundary.

LLM judges scale open-ended evaluation but exhibit position, verbosity, self-enhancement, and reasoning biases Zheng et al. [2023]; G-Eval likewise reports both improved human correlation and possible evaluator bias Liu et al. [2023]. We therefore reserve an external judge for cases where deterministic answer equivalence returns “unknown,” never for grounding. All judge requests use a frozen JSON schema and are retained for audit. This design reduces judge scope but does not eliminate the need for future human validation.

3 The STRIVE Metrics

3.1 Notation and Denominators

Let models be indexed by m and tasks by $i \in \{1, \dots, N\}$. A trajectory contains the problem x_i , reference answer y_i , visible model actions, successful or failed environment observations, and the declared final answer $\hat{y}_{mi}$. Let $A_{mi} = 1$ when the controller receives a nonempty explicit final-answer action and $A_{mi} = 0$ otherwise. For valid trajectories, visible steps are indexed by $t \in \{1, \dots, L_{mi}\}$; here L_{mi} is trajectory length, while T is reserved for token counts. Process metrics use only classified non-answer steps, whose count is K_{mi} . If $A_{mi} = 0$ or $K_{mi} = 0$, process metrics are left undefined rather than filled with zero.

We report two denominators for outcome metrics. Operational rates keep every attempted task in the denominator:

$$\bar{Z}_m^{\text{op}} = \frac{1}{N} \sum_{i=1}^N A_{mi} Z_{mi}, \quad \bar{Z}_m^{\text{val}} = \frac{\sum_i A_{mi} Z_{mi}}{\sum_i A_{mi}}, \quad (1)$$

for $Z \in \{C, G, V\}$, where C is correctness, G is grounding, and V is verified success. The valid-only rate $\bar{Z}_m^{\text{val}}$ is used only as a diagnostic when $\sum_i A_{mi} > 0$. Coverage $\bar{A}_m = N^{-1} \sum_i A_{mi}$ is reported separately. This avoids survivorship bias in headline outcomes and avoids double-counting coverage by multiplying an already operational rate by $\bar{A}_m$ again.

3.2 Correctness, Grounding, and Verified Success

Correctness asks whether the declared final answer matches the reference answer. The deterministic checker first applies normalized exact matching, then mathematical parsing/equivalence, and finally symbolic simplification using SymPy Meurer et al. [2017]. It returns $C_{mi}^{\text{sym}} \in \{0, 1, \perp\}$, where $\perp$ denotes an undecidable parse. Only when $C_{mi}^{\text{sym}} = \perp$ is the frozen judge called, producing $C_{mi}^{\text{judge}} \in \{0, 1\}$ when the call succeeds. For valid trajectories,

$$C_{mi} = \begin{cases} C_{mi}^{\text{sym}}, & C_{mi}^{\text{sym}} \neq \perp, \\ C_{mi}^{\text{judge}}, & C_{mi}^{\text{sym}} = \perp \text{ and judgment succeeds,} \\ 0, & \text{otherwise (reported as unresolved).} \end{cases} \quad (2)$$

Invalid trajectories have operational contribution $A_{mi}C_{mi} = 0$.

Grounding asks a different question: whether the declared final answer is traceable to observed sandbox evidence. For a successful tool action at step t , let $\mathcal{E}_{mit}$ be the answer candidates parsed from stdout: boxed expressions, answer/result assignments, final printed lines, and numeric snippets. Let $\equiv (a, b)$ denote deterministic mathematical equivalence. For valid trajectories,

$$\begin{aligned} G_{mi} &= \mathbb{I}[\exists t \in \{1, \dots, L_{mi}\}, \exists e \in \mathcal{E}_{mit} : \equiv (\hat{y}_{mi}, e) = 1], \\ V_{mi} &= C_{mi}G_{mi}. \end{aligned} \quad (3)$$

The evidence pointer stores the matched step, candidate, and output hash. A matching intermediate output does *not* ground a different final answer: the relation is always between $\hat{y}_{mi}$ and a recorded candidate. Four diagnostic levels refine G : G0, no successful tool evidence; G1, evidence exists but the final answer is not traceable; G2, traceable but incorrect against the reference; G3, traceable and correct. G3 is not claimed to be a second formal proof: it is the conjunction of traceability and reference correctness.

3.3 Hybrid Step Quality

Step quality asks whether the visible process is moving toward a solution. For each classified non-answer step $t \in \{1, \dots, K_{mi}\}$, two PRMs produce scores $p_{mit}^{(1)}$ and $p_{mit}^{(2)}$; their arithmetic mean is p_{mit} . With $p_{mi0} = 0.5$, the bounded PRM change is

$$z_{mit} = \tanh\left(\frac{p_{mit} - p_{mi,t-1}}{0.10}\right). \quad (4)$$

Let $d_{mit} = 1$ if successful code emits a new normalized observation, $r_{mit} = 1$ for a harmful repeat (defined below), and $e_{mit} = 1$ for a code or recorded execution error. The base score combines PRM movement with deterministic execution signals:

$$b_{mit} = 0.45z_{mit} + 0.15d_{mit} - 0.25r_{mit} - 0.50e_{mit}. \quad (5)$$

Only for non-answer steps with $|b_{mit}| < 0.08$ is the critic invoked. Its structured label maps to $s_{mit} \in \{1, 0.25, 0, -0.5, -1\}$ for progressive, neutral-useful, neutral-waste, redundant, and regressive, respectively, and $h_{mit} = b_{mit} + 0.25s_{mit}$. Precedence is deterministic: errors or $h \leq -0.20$ are regressive; otherwise harmful repetition is redundant; $h \geq 0.20$ is progressive; a useful check is neutral-useful; all remaining steps are neutral-waste. For reporting, label utility is $u \in \{1, 0.5, 0, -0.5, -1\}$ in the same order, and

$$Q_{mi} = \text{clip}_{[0,1]}\left(\frac{1 + K_{mi}^{-1} \sum_{t=1}^{K_{mi}} u_{mit}}{2}\right). \quad (6)$$

The weights were fixed during smoke-test development before scoring the final pool. They are an operational rubric, not learned ground truth; PRM agreement and critic ablations are therefore mandatory diagnostics.

3.4 Token Utility and Cost

Token utility asks where visible output tokens are spent relative to the first point at which the answer is available. For a correct trajectory, the solution point t_{mi}^* is the first successful tool step whose

output is equivalent to the declared final answer; if no such tool step exists, t_{mi}^* is the explicit answer step. For incorrect or untraceable trajectories, t_{mi}^* is undefined, so non-answer process tokens are counted as pre-solution waste unless a step is explicitly labeled useful verification.

Visible process tokens before t_{mi}^* are useful when their step label is progressive or neutral-useful and are pre-solution waste otherwise. Non-answer process tokens after t_{mi}^* are post-solution waste. Answer-reporting tokens are retained as a separate component and are not included in the PTU denominator. With useful, pre-waste, and post-waste counts $U, W^{\text{pre}}, W^{\text{post}}$,

$$\text{PTU}_{mi} = \frac{U_{mi}}{\max(U_{mi} + W_{mi}^{\text{pre}} + W_{mi}^{\text{post}}, 1)}. \quad (7)$$

We separately record API input, API output, provider-reported hidden reasoning when available, code, tool output, answer-reporting tokens, and latency. Billable tokens are not double-counted: $T^{\text{bill}} = T^{\text{api,in}} + T^{\text{api,out}}$; code is a subtype of model output and tool output is reported separately. Deployment efficiency is summarized by

$$\text{CPVS}_m = \frac{\sum_i T_{mi}^{\text{bill}}}{\sum_i A_{mi} V_{mi}}, \quad (8)$$

and by Pareto frontiers. Budget curves retrospectively truncate visible API-output prefixes at 512–8192 tokens and require both answer and evidence to be present; they are not fresh generations under hard provider budgets.

3.5 Reasoning-Aware Redundancy

Redundancy asks whether repeated-looking reasoning is harmful or useful. Each classified process step is compared with earlier steps after math normalization (canonical whitespace, variables, and numeric forms), exact matching, SBERT cosine similarity Reimers and Gurevych [2019], and symbolic comparison of extracted expressions. Let ρ_{mit} mark any detected repeat and v_{mit} mark useful verification. At threshold $\tau = 0.85$,

$$\begin{aligned} r_{mit} &= \mathbb{I}[\rho_{mit} = 1 \wedge v_{mit} = 0 \wedge d_{mit} = 0], \\ R_{mi}^{\text{harm}} &= K_{mi}^{-1} \sum_{t=1}^{K_{mi}} r_{mit}. \end{aligned} \quad (9)$$

Useful verification requires successful code, verification language, and output equivalent to the final answer. Exact, semantic, algebraic, useful-verification, and harmful-circular rates are exported separately. Sensitivity is reported for $\tau \in \{0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}$.

An optional per-trajectory composite utility can be computed as

$$E_{mi} = V_{mi} \cdot \text{PTU}_{mi} \cdot Q_{mi} \cdot (1 - R_{mi}^{\text{harm}}). \quad (10)$$

It is included only for user-defined utility analysis and is not used for headline ranking because its multiplicative form encodes preferences.

4 Implementation and Audit Trail

4.1 Controller and Sandbox

All agents receive one protocol: return exactly one fenced `python` action or one fenced `answer` action per turn. A trajectory permits at most four executed tools and two final-answer attempts (six turns), 2,048 output tokens per request, a 90-second request timeout, 150-second aggregate request deadline, and a 300-second trajectory deadline. Models run concurrently in independent lanes, while steps within a trajectory remain sequential. Checkpoints are appended after every problem.

The controller, not the model, creates observations. It rejects placeholder, multiple, repeated, or answer-only-violating actions and never promotes an intermediate tool value to the final answer. The full executable contract appears as Algorithm 1 in the supplementary material.

The Python executor is intended for trusted benchmark mathematics, not adversarial untrusted code. Before execution it permits only math/scientific imports and blocks filesystem, process, network,

Table 1: Final operational results. A is valid-trajectory coverage; C, G, V use all 300 attempts. Q , PTU, and R^{harm} use valid trace-observable attempts. Tok./V is billable tokens per verified solve. GPT-OSS process metrics are intentionally omitted. Best values are bold where “best” is meaningful; lower is better for R^{harm} , Tok./V, and latency.

Agent	A	C	G	V	Q	PTU	R^{harm}	Tok./V	Lat.	P95
GLM-5.2	1.000	0.730	0.620	0.563	0.486	0.255	0.011	3,323	49.2	125.9
GPT-5 Nano	0.933	0.667	0.727	0.610	0.471	0.280	0.136	5,312	10.1	23.2
GPT-OSS-20B	0.867	0.597	0.017	0.010	—	—	—	109,050	8.8	19.5
MiniMax-M3	0.993	0.760	0.473	0.390	0.448	0.271	0.004	8,801	113.6	341.8
Ministral-14B	0.987	0.363	0.190	0.173	0.164	0.100	0.012	9,503	26.4	93.1
Nemotron-3-Nano	0.997	0.617	0.740	0.550	0.416	0.239	0.193	7,375	19.2	52.1

dynamic evaluation, and dunder introspection primitives. Each action runs under `python -I` in a fresh subprocess and temporary directory with a clean environment, 3,072 MB address-space limit, 20-second CPU/tool timeout, 10 MB file limit, and process limit 16. Earlier successful blocks are replayed with `stdout/stderr` muted to provide state; failed blocks are not committed. Code, `stdout`, `stderr`, success, elapsed time, and an evidence hash are stored.

4.2 Evaluators, Failure Policy, and Reproducibility

The PRMs are Math-Shepherd-Mistral-7B and Qwen2.5-Math-PRM-7B, loaded and offloaded sequentially. A frozen Step-3.5-Flash evaluator, accessed through NVIDIA NIM with deterministic classification settings, handled both selective correctness and critic calls in the repaired run. It completed 134/134 correctness and 677/677 critic requests. Grounding never uses this evaluator.

A valid trace must finish with stop reason `explicit_final_answer`, a nonempty answer, and at least one recorded step. Invalid attempts receive operational $C = G = V = 0$; process metrics are missing. We retain six-model $C/G/V$, cost, latency, coverage, and protocol adherence. GPT-OSS-20B is excluded from $Q/PTU/R^{\text{harm}}$: 99.5% of its steps carried provider reasoning, 32.6% of those traces could not be mapped to a visible controller action, and only 1.95% were successful code steps. Treating hidden provider prose as an executed action would fabricate process evidence.

The portable artifact is deduplicated by (m, i) . Later repaired records replace earlier records regardless of answer correctness, preventing best-of- n selection. It contains the frozen problems, raw messages, actions, tool records, PRM scores, judge/critic JSON, metrics, manifests, and SHA-256 checksums.

5 Experiments and Results

5.1 Setup

We select 200 MATH-500 test problems Hendrycks et al. [2021] by deterministic round-robin balancing over subject and level, and 100 English, text-only, single-answer competition-mathematics problems from OlympiadBench He et al. [2024], balanced by subject. Seed 42 freezes selection and three 100-problem batches. The six agents are GLM-5.2, GPT-5 Nano, GPT-OSS-20B, MiniMax-M3, Ministral-14B, and Nemotron-3-Nano-30B-A3B. Each receives one trajectory per problem. Model endpoint identifiers and inference parameters are in the supplement and manifest.

We report operational rates over all 300 attempts per model. Means for trace metrics use valid, observable traces. Problem-level 95% confidence intervals use 5,000 bootstrap resamples. Paired binary comparisons use exact McNemar tests by problem ID with Holm correction McNemar [1947], Holm [1979]; bootstrap rank stability also uses 5,000 problem resamples.

5.2 Accuracy Alone Misstates Operational Quality

Table 1 shows why a single final-answer score is insufficient. MiniMax-M3 is most correct ($C = 0.760$), but only 51.3% of its correct answers are grounded, producing $V = 0.390$. GPT-5 Nano is less correct ($C = 0.667$) but retains 91.5% of correct answers as verified, yielding the highest $V = 0.610$. Nemotron-3-Nano is also less correct than MiniMax-M3 (0.617 versus 0.760) but

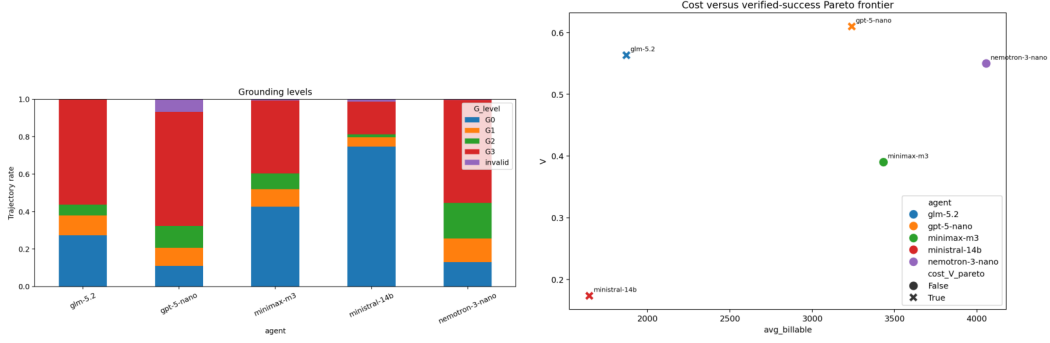


Figure 2: Left: grounding-level composition. G0/G1 identify absent or untraceable evidence; G2 is traceable but incorrect; G3 is traceable and correct. Right: billable-token cost versus verified success for trace-comparable models; crosses mark the Pareto frontier. GPT-OSS is shown only in operational tables because its process trace is not comparable.

more verified (0.550 versus 0.390), because a larger share of its successful answers can be traced to sandbox evidence. The central empirical pattern is therefore not that the ranking is merely reordered, but that correctness and evidential support measure different failure modes.

The paired tests reinforce this distinction. GLM-5.2 and Nemotron are statistically indistinguishable on verified success in the paired test (difference 0.013, Holm $p = 0.749$), while GPT-5 Nano versus GLM-5.2 is also not significant after correction (difference 0.047, Holm $p = 0.477$). In contrast, GPT-5 Nano exceeds MiniMax-M3 by 0.220 verified success (Holm $p = 3.65 \times 10^{-8}$). A paper that reported only C would conclude that MiniMax-M3 is the strongest model; the audit view shows that its answers are often not supported by executable evidence.

The benchmark split further shows that the effect is not confined to one source. On the 200 MATH-500 items, verified success is 0.680 (GPT-5 Nano), 0.650 (GLM), 0.615 (Nemotron), 0.430 (MiniMax), and 0.220 (Ministral). On the 100 OlympiadBench items it drops to 0.470, 0.390, 0.420, 0.310, and 0.080, respectively. These are within-sample diagnostics, not claims about all Olympiad problems, but they show that verified success remains informative when problem style shifts.

5.3 Grounding, Cost, and Latency Trade-offs

Figure 2 exposes the deployment choices hidden by accuracy. The left panel separates absent evidence, untraceable evidence, traceable-but-wrong evidence, and traceable-and-correct evidence. GPT-5 Nano and Nemotron have the largest G3 mass, explaining their strong verified success. MiniMax-M3, despite the highest C , has a much smaller G3 share, so many of its correct answers remain unsupported by the recorded execution trace. GPT-OSS-20B has nontrivial correctness but almost no grounded success, which is why it is retained in operational tables but excluded from process comparisons.

The right panel shows that there is no universal winner under cost. GLM-5.2 has the best token rate (3,323 billable tokens per verified solve) but averages 49.2 seconds because conservative request pacing adds wait. GPT-5 Nano is fastest among trace-comparable systems (10.1 seconds) and has the highest V , but uses 5,312 tokens per verified solve and emits 98.2 mean post-solution tokens. MiniMax-M3 is dominated in the cost- V plane and has a heavy latency tail (P95 341.8 seconds). Nemotron is fast and well grounded, but pays with more repetition. These trade-offs motivate dashboard and Pareto reporting rather than a scalar leaderboard.

5.4 Verified Success Under Token Budgets

Figure 3 asks whether a model reaches a verified answer early in its visible output. At a retrospective 512-token budget, verified success is already 0.533 for GLM, 0.460 for Nemotron, 0.453 for GPT-5 Nano, 0.270 for MiniMax, and 0.167 for Ministral. GPT-5 Nano then rises sharply to 0.587 at 1,024 tokens and saturates at 0.610 by 2,048. GLM is comparatively flat: most of its verified solves appear

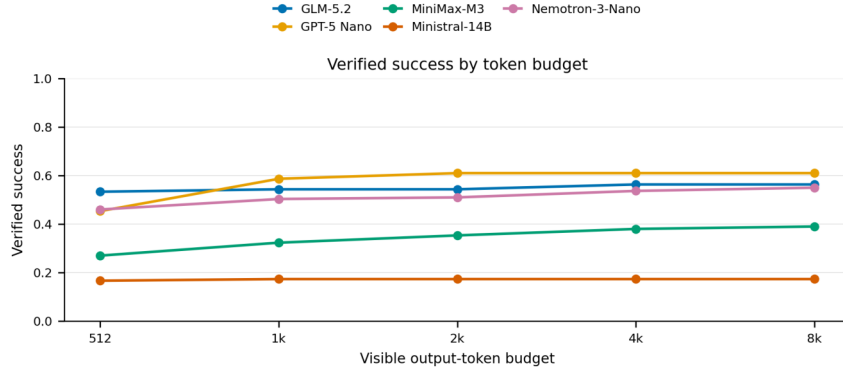


Figure 3: Verified success under retrospective visible output-token budgets. Each point requires a correct final answer and traceable evidence.

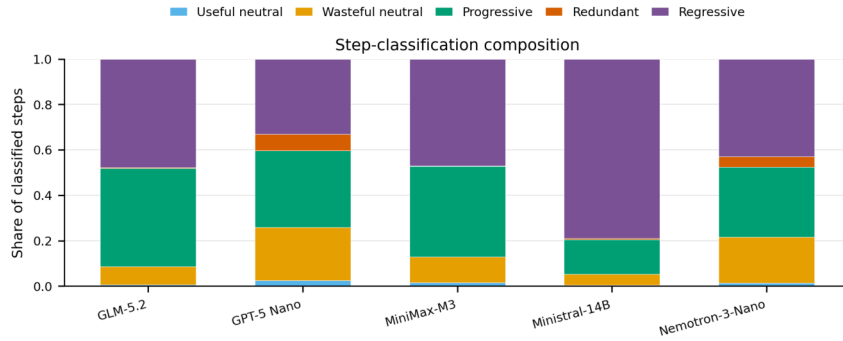


Figure 4: Step-classification composition for the five trace-comparable models. Bars show the share of observable steps in each process label.

early, which explains why it has the best tokens-per-verified-solve despite not having the highest final V .

The curves also diagnose model-specific inefficiency. MiniMax-M3 continues improving through 8,192 tokens, indicating that some verified solves require longer visible traces. Nemotron improves more gradually and remains below GPT-5 Nano, even though its final grounding rate is slightly higher; its issue is less evidence availability than correctness and repetition. Ministral-14B remains low at every budget, so its weakness is not simply that it needs more room to finish. These curves are descriptive prefix analyses over completed trajectories, not counterfactual hard-budget generations, but they identify where future budgeted runs should focus.

5.5 Step Quality and Token Utility

Figure 4 explains how Q and PTU differ from final success. GLM-5.2 has the highest Q (0.486) because a large share of its observable steps are classified as progressive and very few are redundant. GPT-5 Nano has the highest PTU (0.280), meaning that more of its visible process tokens occur in steps judged useful before the solution point, but it also spends visible tokens after the answer has effectively been found. MiniMax-M3 shows strong correctness but a lower verified rate, consistent with many steps being locally plausible without producing traceable final evidence. Ministral-14B has the weakest Q and PTU, matching its low verified success.

This analysis is deliberately cautious about process supervision. The two PRMs agree only moderately over 4,383 paired steps (Spearman $\rho = 0.379$, mean absolute difference 0.333), so neither PRM is treated as ground truth. The selective critic is invoked only near a decision boundary and changes mean Q by 0.004–0.035 across models; all 677 selected calls succeeded. Thus the step-quality metric should be read as a calibrated diagnostic signal, not as a claim that the framework has solved human-level reasoning-step annotation.

5.6 Reasoning-Aware Redundancy

Figure 5 evaluates whether the redundancy conclusion is an artifact of one similarity threshold. At the default $\tau = 0.85$, harmful redundancy is low for GLM-5.2 (0.011), MiniMax-M3 (0.004), and Ministral-14B (0.012), but much higher for GPT-5 Nano (0.136) and Nemotron (0.193). The sensitivity curve preserves this ordering across $\tau = 0.70$ –0.95: stricter thresholds reduce absolute rates, but Nemotron and GPT-5 Nano remain the most repetitive.

This matters because STRIVE does not label every repeated-looking step as waste. Repetition is harmful only when it repeats prior reasoning without new progress and without a useful verification role. A model that checks a computed answer can therefore receive neutral-useful credit rather than a redundancy penalty. The threshold analysis supports the claim that the high-redundancy models are not artifacts of one arbitrary cutoff, while the low-redundancy models remain low across a broad range.

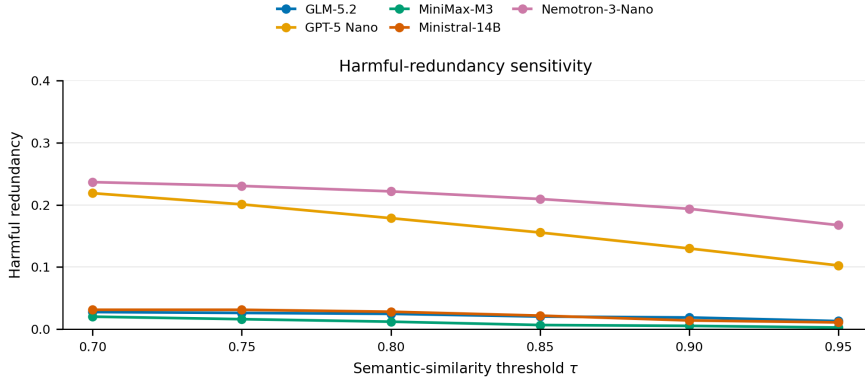


Figure 5: Harmful-redundancy sensitivity to the semantic-similarity threshold τ . The axis is restricted to the observed range to make threshold differences visible.

6 Generalization, Limitations, and Conclusion

6.1 Limitations

First, the empirical study contains one sampled generation per model–problem pair in mathematics; it does not demonstrate non-math validity or variance across prompts, seeds, and endpoint versions. Second, the sandbox restricts common unsafe primitives and resources but is not a security boundary for hostile code. Third, PRM/critic process labels lack a new human-labeled calibration set. Moderate PRM agreement and the critic ablation quantify disagreement but do not establish semantic validity. Fourth, grounding proves traceability to observed output, not that the program embodies the intended derivation; G2 explicitly captures traceable wrong outputs, and even G3 is not a formal proof. Fifth, provider-reported token categories differ. Hidden reasoning was unavailable in these endpoints, so PTU concerns visible process output. Sixth, threshold sensitivity is extensive for redundancy but not yet a full factorial sensitivity analysis over every hybrid-step weight. Finally, judge completion was 100%, but evaluator-model bias remains possible; the released decisions permit later human audit.

6.2 Conclusion

Sandboxed-agent evaluation should distinguish getting an answer from earning confidence in how it was obtained. STRIVE makes that distinction operational through correctness, deterministic provenance, verified success, hybrid step quality, solution-point token utility, harmful redundancy, reliability, and latency. Across 1,800 attempts, these dimensions change the practical interpretation of model quality: the most accurate model is not the most verified, the most grounded is not the least redundant, and the fastest is not the cheapest per verified solve. The framework therefore recommends a dashboard, uncertainty intervals, and Pareto frontiers. Its main scientific claim is not a universal winner, but an auditable method for discovering why agents that look similar under accuracy behave differently in execution.

References

- Wenhu Chen, Xueguang Ma, Xueguang Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=YfZ4ZPt8zd>.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799, 2023.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3828–3850, 2024. doi: 10.18653/v1/2024.acl-long.211.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 34, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html>.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=i04LZibEqW>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, 2023. doi: 10.18653/v1/2023.emnlp-main.153.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 37, 2024. doi: 10.52202/079017-2365.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondrej Certik, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, et al. SymPy: Symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017. doi: 10.7717/peerj-cs.103.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=fibxvahvs3>.

- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9426–9439, 2024a. doi: 10.18653/v1/2024.acl-long.510.
- Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback. In *International Conference on Learning Representations*, 2024b.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024, 2025. doi: 10.18653/v1/2025.acl-long.50.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 36, 2023.